Remerciement

Nous tenons à exprimer notre plus sincère gratitude à Monsieur le Recteur de l'**ISPM (Institut Supérieur Polytechnique de Madagascar)**, pour son soutien moral et institutionnel. Son intérêt pour notre travail ainsi que les conditions qu'il a su mettre en place ont grandement facilité la réalisation de cet ouvrage. Qu'il trouve ici l'expression de notre profond respect et de notre reconnaissance.

Nous remercions aussi l'ensemble du **corps enseignant de l'ISPM**, pour la qualité des enseignements dispensés, pour leur engagement et pour les échanges enrichissants qui ont marqué notre formation.

À nos **familles et à nos proches**, nous adressons un remerciement particulier pour leur patience, leur compréhension et leur appui constant, sans lesquels ce projet n'aurait pu voir le jour.

Enfin, nous témoignons notre reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à l'élaboration de ce travail.

Introduction

À l'ère du numérique, la quantité de documents numériques ne cesse d'augmenter. Dans de nombreux domaines administratif, éducatif, juridique ou professionnel, les utilisateurs doivent traiter une grande variété de fichiers contenant des informations parfois complexes ou volumineuses. L'analyse manuelle de ces documents peut être longue et source d'erreurs. C'est dans ce contexte que les technologies d'intelligence artificielle offrent des solutions innovantes pour automatiser et faciliter le traitement des documents.

Ce projet s'inscrit dans cette dynamique en proposant le développement d'une application intelligente capable d'analyser automatiquement des documents fournis par l'utilisateur. Qu'il s'agisse de fichiers PDF, de documents Word ou d'images scannées, le système est conçu pour extraire le contenu textuel, repérer les éléments clés et fournir une aide à la lecture et à la compréhension du document. L'application vise ainsi à offrir un gain de temps considérable, tout en facilitant l'accès rapide à l'information utile.

Une fois le texte extrait, l'intelligence artificielle intervient pour assister l'utilisateur dans l'analyse. Elle permet, par exemple, de repérer des passages importants, de générer un résumé global du document, ou encore de répondre à des questions simples basées sur son contenu. L'approche adoptée reste volontairement générale, sans entrer dans des traitements linguistiques complexes, afin d'assurer la robustesse et la simplicité du système.

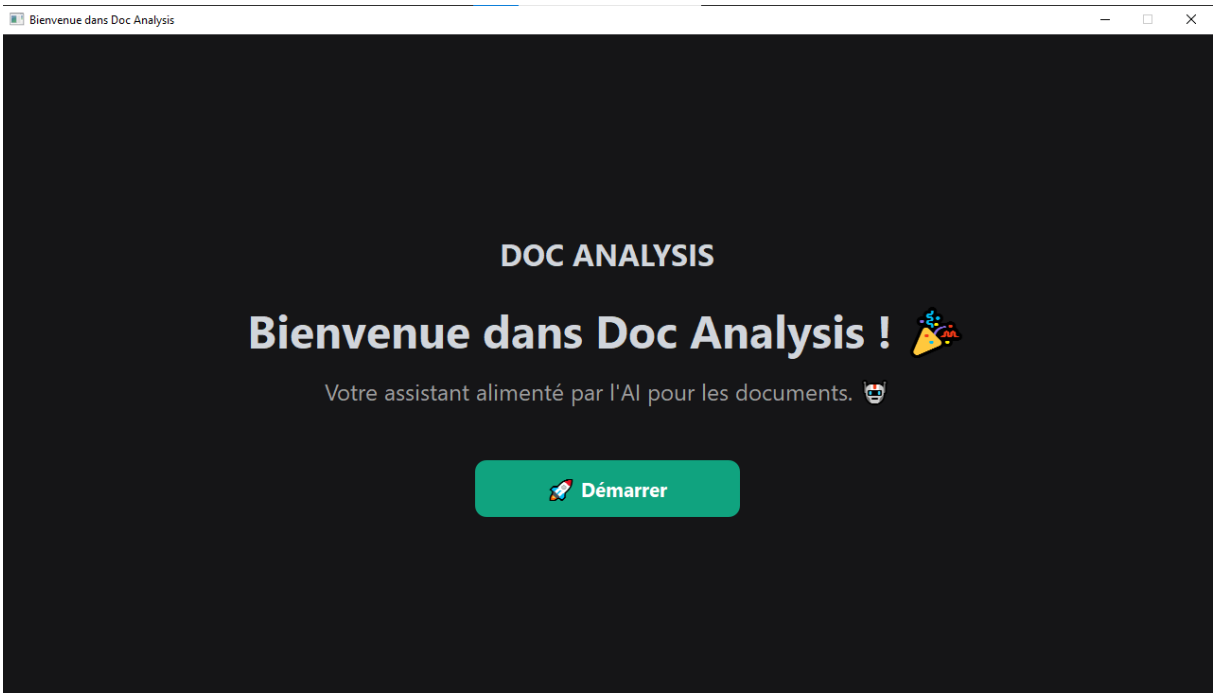
Ce projet représente un défi à la fois technique et pratique, mobilisant plusieurs compétences en développement logiciel, en intégration d'outils d'IA, et en traitement de documents. Il vise à offrir une solution accessible, efficace et polyvalente, adaptée à des besoins concrets d'analyse documentaire dans différents contextes.

Chapitre 1 : Frontend

Notre application Doc Analyses a été développée dans un premier temps sous forme d'application Desktop (pour ordinateur). Pour construire son interface graphique (frontend), nous avons utilisé le langage Python et le framework PyQt5.Ce choix technique nous permet de proposer une interface moderne, légère, et adaptée à notre cas d'usage.

Dans cette partie, nous allons vous présenter les différentes interfaces de l'application ainsi que les fonctionnalités principales qui ont été mises en place.

Page d'accueil:

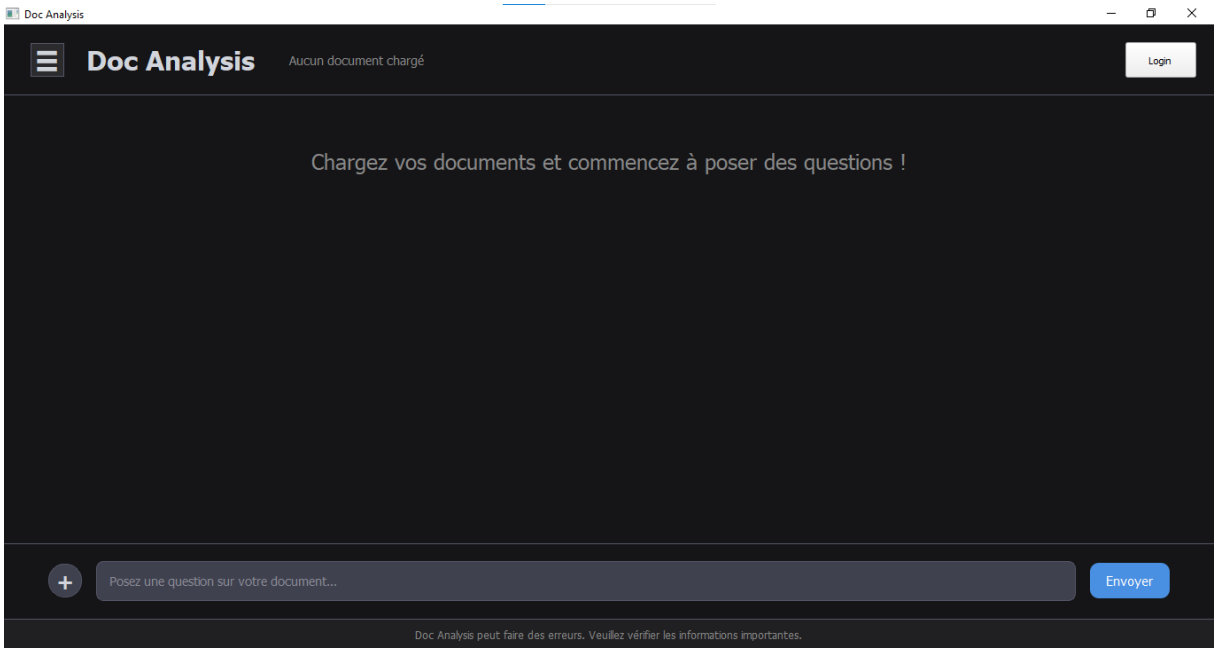


L'ouverture de l'application affiche une page d'accueil simple et épurée.
Cette première page sert d'introduction à l'expérience utilisateur. Elle contient :

- Le titre du projet : **Doc Analyses**
- Une phrase d'accroche permettant d'expliquer brièvement l'objectif de l'application
- Un bouton **Démarrer** permettant à l'utilisateur d'accéder à la suite

Cette approche permet de guider l'utilisateur dès les premières étapes de prise en main de l'outil

Page principale



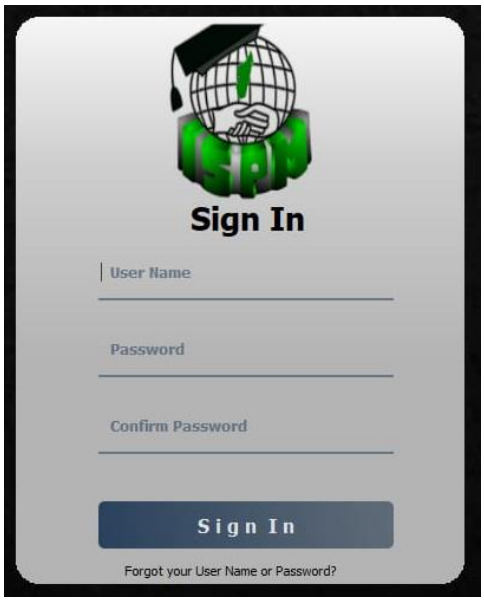
En cliquant sur le bouton **Démarrer**, l'utilisateur accède à la page principale de l'application. C'est l'espace de travail central, où se déroulent les échanges entre l'utilisateur et l'intelligence artificielle.

Cette page est structurée en plusieurs zones fonctionnelles clairement identifiées :

- **L'en-tête :** elle contient la barre de menu, le titre de la discussion en cours, et des informations sur l'état de la session.
- **Zone de discussion :** ici s'affichent les interactions entre l'utilisateur et l'IA. Les messages sont organisés de manière claire, facilitant la lecture du fil de discussion.
- **Zone de saisie :** cette zone permet à l'utilisateur de saisir du texte ou d'envoyer de nouveaux fichiers à analyser. Elle comprend : un bouton « + » pour l'ajout de fichiers, un champ de saisie pour écrire du texte, un bouton Envoyer pour soumettre la demande à l'IA.
- **Pied de page :** cette zone peut contenir des informations complémentaires ou de navigation.

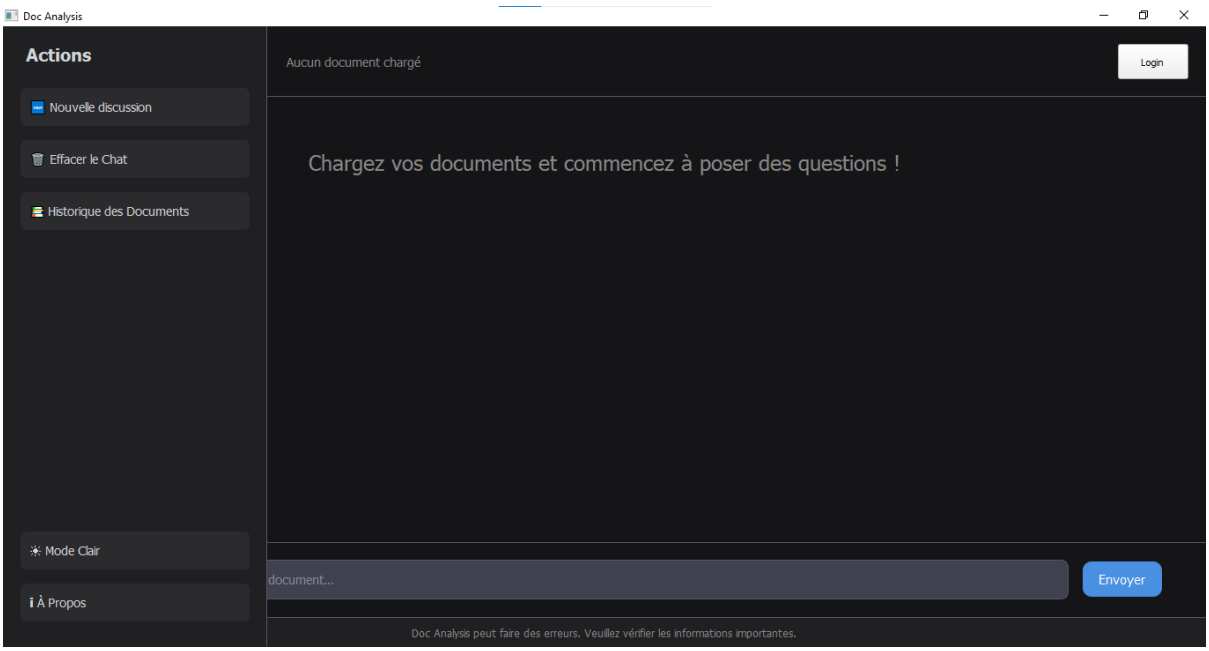
- **Login** : L'utilisateur peut se connecter via son compte e-mail pour améliorer son expérience. Cela lui permet de :

    - Accéder à ses documents personnels
    - Sauvegarder et retrouver l'historique
    - Personnaliser l'application : thèmes, préférences, paramètres
    - Accéder à des fonctionnalités avancées



Grâce à cette interface, l'utilisateur peut interagir de manière intuitive avec l'IA pour obtenir une analyse personnalisée de son document.

Barre de menu



La barre de menu, située en haut de l'interface, offre un certain nombre de fonctionnalités supplémentaires permettant de personnaliser l'expérience utilisateur :

- **Nouvelle discussion** : permet de démarrer une nouvelle session de chat, en réinitialisant l'espace de discussion.
- **Effacer le chat** : permet de supprimer l'ensemble de la discussion en cours.

- **Historique des documents** Cette fonctionnalité permet à l'utilisateur de **consulter tous les documents qu'il a chargés** dans l'application et de **suivre toutes les modifications effectuées** sur chacun d'eux
- **Mode clair / sombre** : l'utilisateur peut basculer entre un mode clair et un mode sombre, selon ses préférences visuelles.
- **À propos** : affiche des informations sur l'application (version, objectifs, technologies utilisées) et sur les membres du groupe ayant contribué au projet

Chapitre 2 : Backend

1. Extraction de texte

Dans le script python, on trouve des outils pour extraire automatiquement du texte et des images depuis différents types de fichiers (PDF, DOCX, PPTX, XLSX, images, TXT). Il s'agit d'une solution complète pour analyser des documents dans un contexte d'archivage, d'indexation ou d'analyse automatisée.

Le script est organisé en différentes sections, chacune responsable d'un format de fichier particulier. Il utilise plusieurs bibliothèques populaires.

- Fitz (PyMuPDF) pour lire les fichiers PDF
- Docx pour les fichiers Word
- Pptx pour les présentations PowerPoint
- Openpyxl pour les fichiers Excel
- PIL et pytesseract pour faire de l'OCR sur des images
- Os pour la gestion de fichiers système

1.1. Extraction de texte brut (TXT)

```
# ———————— TXT ————————
def extract_txt(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        return f.read()
```

* Ouvre un fichier texte en lecture ('r') avec l'encodage UTF-8.

* Retourne tout le contenu sous forme de chaîne

1.2. PDF : Extraction depuis des images

* Crée un dossier images si besoin.

* Pour chaque page du PDF :

- o On récupère la liste des images avec get_images(). o Chaque image est extraite et convertie si nécessaire (selon sa transparence).
- o L'image est enregistrée sous forme de .png avec un nom clair (par page et par image).

```python
# ———————— PDF: Images ————————
def extract_images_from_pdf(pdf_path, output_dir="images"):
    doc = fitz.open(pdf_path)
    os.makedirs(output_dir, exist_ok=True)

    for page_index in range(len(doc)):
        page = doc[page_index]
        image_list = page.get_images(full=True)

        if image_list:
            print(f"[Page {page_index}] Found {len(image_list)} image(s).")
        else:
            print(f"[Page {page_index}] No images found.")

        for image_index, img in enumerate(image_list, start=1):
            xref = img[0]
            pix = fitz.Pixmap(doc, xref)

            if pix.n - pix.alpha > 3:
                pix = fitz.Pixmap(fitz.csRGB, pix)

            image_filename = os.path.join(output_dir, f"page_{page_index}_img_{image_index}.png")
            pix.save(image_filename)
            pix = None
```

1.3. Fichier Word .docx

* Ouvre le fichier Word avec Document ().

* Parcourt chaque paragraphe et récupère le texte.

*Le texte est rassemblé avec des sauts de ligne.

```python
# ———————— DOCX ————————
def extract_text_from_docx(file_path):
    doc = Document(file_path)
    return '\n'.join(p.text for p in doc.paragraphs)
```

Extracte texte de PDF •

Utilise la bibliothèque fitz pour lire les pages du fichier PDF.

- Utilise page.get_text() pour extraire le texte de chaque page.
- Combine les textes de toutes les pages.

```python
def extract_text_from_pdf(pdf_path):
    try:
        doc = fitz.open(pdf_path)
        content_parts = []
        for page in doc:
            content_parts.append(page.get_text())
        doc.close()
        return "\n".join(content_parts)
    except Exception as e:
        return f"Erreur lors de l'extraction du texte PDF : {e}"
```

1.4. Analyse de fichiers Excel

Lecture de cellules Excel .xlsx 🛠
Outil utilisé : openpyxl

Le texte présent dans chaque cellule est lu, feuille par feuille. Chaque ligne est convertie en texte brut, ce qui est utile pour des bases de données ou des tableaux de suivi.

```python
def extract_text_from_xlsx(file_path):
    try:       You, 3 days ago • Merged code
        wb = openpyxl.load_workbook(file_path)
        text = []
        for sheet_name in wb.sheetnames:
            sheet = wb[sheet_name]
            text.append(f"=== Feuille : {sheet_name} ===")
            for row in sheet.iter_rows(values_only=True):
                row_text = [str(cell) if cell is not None else '' for cell in row]
                if any(cell.strip() for cell in row_text):
                    text.append(' | '.join(row_text))
        return '\n'.join(text)
    except Exception as e:
        return f"Erreur lors de l'extraction du texte XLSX : {e}"
```

📽 1.5. Extraction depuis des PowerPoint

Lecture des diapositives .pptx
Outil utilisé : python-pptx

Les titres et textes de chaque diapositive sont lus. Un séparateur "=== Diapositive X ===" est ajouté pour repérer la structure.

```python
def extract_text_from_pptx(file_path):
    try:
        prs = Presentation(file_path)
        text_runs = []
        for slide_num, slide in enumerate(prs.slides, 1):
            text_runs.append(f"=== Diapositive {slide_num} ===")
            for shape in slide.shapes:
                if hasattr(shape, "text") and shape.text.strip():
                    text_runs.append(shape.text)
        return '\n'.join(text_runs)
    except Exception as e:
        return f"Erreur lors de l'extraction du texte PPTX : {e}"
```

Fonctionnalités de extractors.py et Leur Utilité

Ce document présente les fonctionnalités du fichier extractors.py. Chaque section détaille aussi en quoi ces fonctionnalités peuvent aider les utilisateurs dans des situations concrètes.

☑ Fonctionnalités Déjà Intégrées dans l'Application

- Extraction multi-formats :

Permet de lire et analyser de nombreux types de documents (PDF, Word, Excel, images, emails, archives, etc.) sans avoir besoin de logiciels différents.
Exemple : un juriste peut comparer un contrat en PDF, un tableau Excel de factures et un mail lié à une affaire.

- OCR amélioré (Reconnaissance Optique de Caractères) :

Récupère du texte à partir d'images ou de scans, même de qualité moyenne.
Exemple : un étudiant peut transformer ses notes manuscrites scannées en texte éditable.

- Extraction de métadonnées :

Fournit des informations essentielles sur les fichiers (taille, dates, auteur, nombre de pages, hachages de sécurité).
Exemple : un archiviste peut rapidement vérifier qui a créé un document et quand.

- Analyse de texte :

Donne des statistiques : nombre de mots, phrases, lisibilité, langue détectée, mots fréquents.
Exemple : un rédacteur vérifie si son texte est accessible à un large public.

- Traitement par lots :

Analyse automatiquement plusieurs fichiers ou un dossier complet.
Exemple : une entreprise traite en une fois tous ses rapports financiers annuels.

- Exportation des résultats :

Les résultats peuvent être sauvegardés en JSON, CSV, TXT, HTML ou Excel.
Exemple : un analyste exporte les données vers Excel pour les comparer à d'autres sources.

- Extraction de la structure des documents :

Identifie titres, paragraphes, listes, tableaux, images et liens dans un PDF ou DOCX.
Exemple : un étudiant récupère uniquement les titres d'un cours pour préparer son plan de révision.

- Validation et contrôle qualité :

Détecte les erreurs d'encodage ou artefacts OCR, calcule un score de qualité et propose des recommandations.
Exemple : un chercheur est alerté si un scan est illisible et doit être rescanné.

Fonctionnalités avancés :

- Extraction avancée de tableaux PDF :

Récupère des tableaux structurés avec Tabula.
Exemple : un comptable extrait automatiquement les tableaux d'un relevé bancaire.

- Extraction des formulaires PDF :

Récupère les champs remplis dans des formulaires interactifs.
Exemple : une entreprise collecte les réponses à un questionnaire PDF.

- Transformations du contenu :

Convertit le texte brut en JSON structuré, Markdown ou XML.
Exemple : un développeur génère un fichier JSON pour l'intégrer à une API.

- Fusion des résultats d'extraction :

Permet de concaténer tous les textes ou de produire un résumé global.
Exemple : un journaliste obtient un résumé global de plusieurs rapports.

- Rapports HTML interactifs :

Produit des comptes rendus lisibles dans un navigateur.
Exemple : un professeur publie un rapport d'analyse sous forme de page HTML interactive.

- Extraction récursive depuis un répertoire :

Analyse automatiquement tous les fichiers d'un dossier avec filtres.
Exemple : un chercheur charge une archive ZIP de documents scientifiques et obtient une base exploitable.

2. Infrastructure Cloud et API

2.1. Introduction

Dans le cadre de notre projet intitulé *Doc Analysis*, nous avons mis en place une architecture simple mais puissante permettant d'analyser différents types de documents (à la fois texte, PDF, Word, images, etc.) grâce à l'intelligence artificielle. Pour ce faire, nous avons utilisé des outils modernes comme Google Colab, FastAPI et des modèles de traitement de langage. Ce chapitre est consacré à l'explication de cette infrastructure technique. Nous parlerons de Google Colab, des API (notamment FastAPI), et de plusieurs concepts utiles pour comprendre l'ensemble du système.

2.2. Google Colab : Définition et Fonctionnement

2.2.1. Qu'est-ce que Google Colab ?

Google Colab (abréviation de "Colaboratory") est un service gratuit fourni par Google permettant d'exécuter du code Python dans un environnement cloud. Il s'agit en fait d'un carnet Jupyter (ou notebook) exécutable à distance, qui offre un accès rapide et simple à des ressources de calcul comme le CPU ou le GPU, sans configuration locale.

2.2.2. Comment fonctionne Google Colab ?

Colab fonctionne sur le même principe qu'un notebook Jupyter classique : on écrit du code dans des cellules que l'on peut exécuter individuellement. La différence majeure est que tout tourne sur des serveurs distants, ce qui permet d'éviter l'installation de bibliothèques lourdes ou la configuration de l'environnement Python.

Il suffit d'avoir un compte Google pour y accéder, puis de créer un nouveau carnet via l'adresse suivante : https://colab.research.google.com.

2.2.3. Pourquoi avons-nous utilisé Google Colab ?

Dans notre projet *Doc Analysis*, Google Colab nous a permis de :

- Travailler à plusieurs sur le même notebook.
- Installer rapidement toutes les dépendances nécessaires (comme FastAPI, FAISS, transformers, etc.).
- Lancer notre modèle IA sans avoir besoin de matériel local puissant.
- Profiter d'une exécution en ligne gratuite (même avec CPU).

Nous avons fait le choix du **CPU** plutôt que du GPU pour deux raisons principales :

- Le CPU est **gratuit** sur une durée illimitée, alors que l'utilisation du GPU est limitée dans le temps.

- Le modèle utilisé n'étant pas trop lourd, les performances en CPU restaient **acceptables**.

2.3. L'API dans le projet : FastAPI

2.3.1 Qu'est-ce qu'une API ?

Une API (Application Programming Interface) est un ensemble de fonctions permettant à deux logiciels de communiquer entre eux. Dans notre cas, l'API sert de pont entre l'utilisateur (interface PyQt) et le moteur IA (exécuté dans Colab).

2.3.2 FastAPI : pourquoi ce choix ?

FastAPI est un framework Python moderne qui permet de créer rapidement des APIs web performantes et faciles à documenter. Il est rapide, compatible avec OpenAPI, et permet une exécution asynchrone du code. Comparé à d'autres frameworks populaires comme Flask (plus minimaliste) ou Django REST Framework (plus complet mais plus complexe), FastAPI offre un excellent compromis entre simplicité, performance et évolutivité.

Voici un extrait du code de notre API :

```
from fastapi import FastAPI from
pydantic import BaseModel

app = FastAPI()

class Query(BaseModel):
    question: str

@app.post("/ask/") async def
ask_question(q: Query):
    reponse = traiter_question(q.question)  # appel au moteur IA
return {"answer": reponse}
```

Ce code permet de recevoir une question depuis le frontend, la traiter et renvoyer une réponse.

2.3.3 Comment est-elle utilisée dans Doc Analysis ?

- L'utilisateur choisit un document via l'interface graphique (PyQt).

- Le document est envoyé à Colab (ou préalablement stocké sur GitHub).
- L'API interroge le moteur IA avec la question posée.
- La réponse est renvoyée et affichée dans l'interface PyQt.

Cette approche a l'avantage de séparer clairement le frontend (interface utilisateur) et le backend (traitement).

2.4. Organisation des fichiers et accès GitHub

Pour faciliter le déploiement, nous avons stocké certains fichiers sur un dépôt GitHub privé. Le notebook Colab télécharge automatiquement :

- Le fichier main.py contenant les fonctions d'extraction de texte depuis PDF, DOCX, images...
- Le document à analyser (fichier .pdf, .docx, etc.)

Le téléchargement s'effectue via le code suivant :

```
import requests

def download_from_github(path, save_as):
    url = f"https://raw.githubusercontent.com/ton-utilisateur/nom-du-repo/main/{path}"
headers = {"Authorization": f"token {GITHUB_TOKEN}"}      r = requests.get(url,
headers=headers)      with open(save_as, 'wb') as f:          f.write(r.content)
```

Ce mécanisme nous permet de conserver une version centralisée et privée du projet. Nous avons préféré GitHub privé à d'autres solutions comme Google Drive car il permet un meilleur contrôle des accès, une intégration directe avec les notebooks Colab via API, ainsi qu'une gestion de version efficace du code et des documents.

2.5. Utilisation de ngrok pour l'accès distant

Puisque Colab ne permet pas d'accéder directement à localhost depuis l'extérieur, nous avons utilisé **ngrok**, un outil qui crée une URL publique vers une application web locale.

Voici comment on l'intègre dans le projet :

```
from pyngrok import ngrok public_url
= ngrok.connect(8000) print("API
accessible :", public_url)
```

Cela permet à l'application PyQt d'appeler l'API FastAPI en ligne, même si elle tourne dans Colab.

2.6. Conclusion sur l'API et transition vers le modèle IA

La mise en place de l'API via FastAPI, connectée à notre infrastructure dans Google Colab, a constitué une étape essentielle pour permettre à l'utilisateur d'interagir avec le moteur d'analyse documentaire de manière fluide et sécurisée. Ce découplage entre le traitement en backend et l'interface utilisateur en frontend assure modularité et évolutivité.

Après avoir établi ce canal de communication entre l'utilisateur et le système, la prochaine section se concentrera sur le cœur même de l'intelligence artificielle du projet : le modèle utilisé pour interpréter les documents, ainsi que les techniques de fine-tuning qui permettent d'adapter ce modèle à des cas d'usage spécifiques. Cette nouvelle partie viendra approfondir les capacités de compréhension sémantique que nous avons brièvement évoquées dans ce chapitre.

3. Choix du modèle et méthode de fine-tuning

Utilisation d'un Large Language Models (LLM) :

Le projet consiste à développer une application permettant à un utilisateur de poser des questions sur ses propres documents(PDF, Word, TXT) et d'obtenir des réponses uniquement basées sur ces documents. Contrairement à un chatbot généraliste, ce système est limité aux informations contenues dans les documents fournis.

1. Pourquoi utiliser un modèle pré-entraîné?

Créer un modèle de langage à partir de zéro nécessiterait :

- Des milliers de documents pour l'entraînement,
- Des GPU puissants et des ressources informatiques très élevées,
- Des semaines voire des mois d'entraînement,
- Une expertise avancée en deep learning.

L'utilisation d'un modèle pré-entraîné comme Mistral permet de réutiliser un modèle déjà optimisé , tout en l'adaptant à notre cas : répondre sur la base de documents spécifiques.

2. Choix du Modèle : Mistral-7B

Mistral est un LLM(Large Language Model) basé sur l'architecture « Transformer ».Ce n'est pas un réseau de neurones classique : c'est une pile de **blocs Transformer** très optimisés

2.1 Présentation de Mistral

Nous avons choisi le modèle **Mistral 7B**, développé par l'entreprise française Mistral AI et disponible en open source. Il s'agit d'un modèle de langage performant, rapide et léger par rapport à d'autres modèles de même catégorie.

Caractéristique	**Détails**
Nom du modèle	Mistral-7B-v0.1
Taille du modèle	7 milliards de paramètres
Licence	Apache 2.0 (open source)
Langue principale	Multilingue
Avantages	Léger, rapide, adaptable

2.2 Pourquoi Mistral ?

Nous avons sélectionné Mistral pour plusieurs raisons :

- Il est entièrement open source, donc gratuit à utiliser et modifiable.

- Il est optimisé pour les performances, même sur un environnement limité comme Google Colab.
- Il a été conçu pour être facilement fine-tunable, c'est-à-dire qu'on peut l'adapter à un domaine spécifique comme la lecture de documents administratifs ou techniques.

3. Comment Mistral fonctionne (technique) ?

Documents → Vectorisation → Base vectorielle → Question → Recherche → Mistral → Réponse

3.1. Vectorisation des mots – Embeddings :

Un ordinateur ne comprend que des nombres. La vectorisation consiste à transformer chaque mot ou (token) en « embedding » : un vecteur multidimensionnel.

Exemple simplifié :

- « chat » → [0.82, -0.12, 0.47, …]
- « chien » → [0.80, -0.15, 0.45, …]
- « voiture » → [0.05, 0.90, -0.30, …]

Les embeddings permettent au modèle de comparer des mots entre eux et de comprendre des relations sémantiques.

3.2. Réseau « Transformer »:

Mistral repose sur l'architecture « Transformer », qui comprend :

- **Self-Attention :** Chaque mot "pèse" les autres mots du contexte. Le but est de calculer quelles parties sont importantes pour chaque mot.
- **Couches profondes (Deep Layers) :** Plusieurs couches de neurones transforment les embeddings pour extraire des représentations complexes.
- **Softmax final :** Chaque mot du vocabulaire reçoit une probabilité d'être le mot suivant. Le mot avec la probabilité la plus élevée est choisi pour générer le texte.

3.3. Descente de gradient et entraînement du modèle :

Bien que Mistral soit pré-entraîné, il est important de comprendre comment un tel modèle apprend:

- **Fonction de perte (Loss function):** L'objectif est de minimiser l'erreur entre le mot prédit par le modèle et le mot réel du corpus.
-

Exemple : Si la phrase est « Je bois du café » et que le modèle prédit « thé », la fonction de

perte mesure l'écart.

- **Backpropagation**: L'erreur calculée est "propagée en arrière" à travers toutes les couches du réseau.

- **Descente de gradient :** Chaque paramètre du réseau (poids des neurones) est ajusté dans la direction qui réduit la perte.

3.4. Application de Mistral dans notre projet

- L'utilisateur charge des fichiers PDF, Word ou TXT.
- Chaque document est nettoyé et découpé en chunks de 300–500 mots.
- Chaque chunk est transformé en embedding via un modèle de type `SentenceTransformer`.
- Les embeddings sont stockés dans une base vectorielle (FAISS) pour recherche rapide.
- Recherche et génération : L'utilisateur pose une question. La question est vectorisée de la même façon que les documents. Le système recherche les chunks les plus similaires dans la base vectorielle. Ces extraits sont passés avec la question à Mistral, qui génère une réponse contextuelle.
  Exemple de prompt pour Mistral :
  Extraits de documents : "L'électricité est une forme d'énergie résultant du déplacement des électrons. Le courant se mesure en ampères."
  Question posée par l'utilisateur : Qu'est-ce que l'électricité ?
  Réponse générée : L'électricité est une forme d'énergie produite par le déplacement des électrons.

4. Fenêtre d'Historique des Versions

4.1 Présentation Générale

Le fichier version_history_window.py gère l'affichage et la comparaison des différentes versions d'un document au sein du projet. Il propose une interface graphique moderne, basée sur PyQt5, permettant à l'utilisateur de consulter l'historique, de visualiser le contenu d'une version et de comparer deux versions.

4.2 Structure du Module

Le module contient principalement deux classes :

- **DiffHighlighter** : surligneur syntaxique pour la coloration des différences entre versions.
- **VersionHistoryWindow** : fenêtre principale affichant la liste des versions et leur contenu.

4.3 Fonctionnalités Principales

- **Affichage de la liste des versions** : chaque version est présentée avec sa date et un extrait du contenu.
- **Visualisation du contenu** : sélection d'une version pour afficher son contenu complet.
- **Comparaison de versions** : sélection de deux versions pour afficher un diff visuel, avec coloration des ajouts, suppressions et modifications.

- **Thématisation dynamique** : adaptation automatique au thème (clair ou sombre) de la fenêtre parente.

4.4 Détail des Classes

- **DiffHighlighter** : applique des couleurs spécifiques aux lignes ajoutées (vert), supprimées (rouge), inchangées (gris) et aux en-têtes de diff (bleu).
- **VersionHistoryWindow** : gère l'interface utilisateur, la sélection des versions, l'affichage du contenu ou du diff, et l'application du thème.

4.5 Utilisation et Intégration

La fenêtre peut être appelée depuis l'application principale, en lui passant le nom du document et la liste des versions. Un exemple d'utilisation autonome est inclus dans le fichier, avec une classe parent simulant l'application et son thème.

4.6 Points Forts

- Interface ergonomique et moderne.
- Sélection automatique de la dernière version.
- Extensible et personnalisable.
- Cohérence visuelle avec le reste de l'application.

4.7 Conclusion

La fenêtre d'historique des versions est un outil essentiel pour la traçabilité et la gestion documentaire du projet, offrant une expérience utilisateur complète et intuitive.

Chapitre 3 : Opportunités économiques et sociales

Ce projet d'outil présente un fort potentiel économique et peut également avoir un impact social positif à différentes échelles : individuelle, professionnelle et institutionnelle.

1. Opportunités Économiques

1.1 Amélioration de la productivité

De nombreuses personnes perdent du temps à chercher des informations dans des documents longs ou techniques. Ce logiciel permet de :

- gagner du temps dans la lecture de rapports, contrats, thèses, etc.
- automatiser les résumés ou les réponses à des questions fréquentes.

Exemples :

Un avocat peut obtenir un résumé rapide d'un contrat de 40 pages.

Un comptable peut extraire des données-clés d'un rapport financier.

1.2 Outil commercialisable :

Le logiciel peut être développé en produit commercial, accessible : via une application Web (SaaS) avec abonnement mensuel, en licence pour les entreprises ou administrations. Cela ouvre la porte à : la vente de services personnalisés (fine-tuning pour les clients) , des modèles freemium : usage gratuit limité, version premium payante.

1.3 Création d'emplois dans le numérique

Ce projet peut générer des opportunités professionnelles :

- développeurs IA,
- spécialistes en traitement de documents,
- responsables de service client pour la solution.

Il peut aussi être intégré dans un projet entrepreneurial ou incubateur de start-up.

1.4 Réduction des coûts administratifs

Dans les entreprises ou institutions publiques, le traitement manuel des documents coûte cher :

- tri des formulaires,

- analyse de dossiers,
- réponse à des demandes répétitives.

Le logiciel permet d'automatiser ces tâches, réduisant ainsi les coûts de fonctionnement.

2. Opportunités Sociales

2.1 Meilleur accès à l'information :

Beaucoup de personnes ont des difficultés à lire ou comprendre des documents complexes :contrats juridiques , documents administratifs , contenus techniques.

Grâce à ce logiciel, elles peuvent :

- poser des questions simplement sur les textes,
- obtenir des résumés plus faciles à comprendre,
- accéder à l'information sans être expert.

2.2 Inclusion numérique :

Le projet permet de :

- réduire les inégalités d'accès au savoir,
- aider les personnes en situation de handicap (par exemple, dyslexie ou déficience visuelle),
- rendre les contenus plus disponibles

2.3 Application dans l'éducation :

Les élèves, enseignants et étudiants peuvent utiliser ce logiciel pour :

- résumer des cours ou des livres scolaires,
- s'entraîner à poser des questions sur les contenus,
- accélérer la révision pour les examens.
- Éducation et apprentissage autonome

C'est donc un outil pédagogique puissant pour favoriser l'apprentissage.

2.4 Contribution à l'innovation locale :

Ce projet valorise l'usage de technologies open source (comme le modèle Mistral) et peut être :

- intégré dans des contextes africains, francophones ou multilingues,
- utilisé pour numériser et analyser des documents publics (lois, circulaires, etc.).

Conclusion

Notre projet n’est pas seulement une solution technique. C’est aussi :

- un outil économique qui peut être développé en produit commercial,
- un levier social pour améliorer l’accès à l’information, l’éducation et l’inclusion.
- Il peut être utilisé par les entreprises, les écoles, les universités, les gouvernements et les particuliers, et contribuer à un monde plus efficace, juste et connecté.

Opportunités	**Explications**
💼 Productivité	Moins de temps à chercher/lire → gain économique
🧠 Éducation	Aide à l’apprentissage, au résumé, à la compréhension
🤝 Inclusion	Moins de barrières pour lire des documents complexes

Conclusion

Ce projet marque une première étape vers une nouvelle manière d'interagir avec les documents numériques. Grâce à l'intelligence artificielle, nous avons réussi à créer un outil accessible qui facilite la compréhension, la synthèse et l'analyse de fichiers complexes, qu'ils soient au format PDF, Word ou image. En simplifiant cette interaction, nous répondons à un besoin réel : gagner du temps, mieux comprendre l'information, et rendre ces contenus plus accessibles à tous, notamment dans les milieux éducatifs, administratifs et professionnels.

Pour la suite, l'évolution naturelle de Doc Analyzer pourrait passer par une version plus autonome, plus rapide et ouverte à davantage de formats. Une version 2.0 pourrait intégrer des fonctions de résumé vocal ou de traduction, de reconnaissance de structure (titres, paragraphes, tableau) tout en gardant la même simplicité d'usage. L'objectif reste le même : faire de cet assistant un véritable partenaire du quotidien, capable de comprendre et d'expliquer les documents, et d'accompagner l'utilisateur dans son exploration du savoir.

BIBLIOGRAPHIE

✞ Webographie

○ [1] https://colab.research.google.com ○ [2] https://research.google.com/colaboratory/ ○ [3] https://fastapi.tiangolo.com ○ [4] https://github.com/facebookresearch/faiss ○ [5] https://realpython.com/fastapi-python-web-apis/ ○ [6] https://github.com/alexdlaird/pyngrok ○ [7] https://pypi.org/project/pytesseract/ ○ [8] https://docs.github.com/en/rest ○ [9] https://youtube.com/1littlecoder ○ [10] https://huggingface.co ○ [11] https://www.youtube.com/@alejandro_ao ○ [12] https://doc.qt.io/archives/qtforpython-5/ ○ [13] https://w3school.com ○ [14] https://zetcode.com/gui/pyqt5/ ○ [15] https://www.riverbankcomputing.com/static/Docs/PyQt5/ ○ [16] https://pyinstaller.org/en/stable/ ○ [17] https://openclassrooms.com/fr/courses/7162856-apprenez-a-utiliser-git-et-github ○ [18] https://www.codecademy.com/learn/learn-git